

***RTKLIB-B2b* User Manual**

A Real-Time Decoding Module for BDS-3 PPP-B2b in RTKLIB

Written by **Liuchunbo**
Email: liuchunbo24@mailsucas.ac.cn

Version: 0.3(2026.03.02)



**Innovation Academy for Precision Measurement Science and Technology,
Chinese Academy of Sciences**

Contents

1. Introduction	1
2. RTKLIB-B2b Structure, Compilation, and Installation	3
(1) RTKLIB-B2b Software Structure	3
(2) RTKLIB-B2b Software Compilation and Installation	6
3. RTKLIB-B2b Decoder	10
(1) Post-processing Decoder	10
a. Usage	10
b. Options	10
c. Example	10
(2) Real-time Decoder	11
a. Usage	11
b. Options	11
c. Console Commands	12
d. Example	12
(3) Decoding result format description	14
a. Epoch Record	15
b. Satellite Mask 'MASK' Example	16
c. Orbit Corrections and URAI 'ORBIT_URAI' Example	17
d. Differential Code Bias Corrections 'DIFF_CODE_BIAS' Example	17
e. Clock Corrections 'CLOCK' Example	18
4. RTKLIB-B2b Positioning	18
(1) RTKLIB-B2b Post-processing PPP Positioning	18
a. Usage	19
b. Options	19
c. Example	19
(2) RTKLIB-B2b Real-time PPP Positioning	21
a. Usage	21
b. Options	21
c. Console Commands	22
d. Example	22
5. Software Maintenance and Data Sharing	25
(1) Software Maintenance	25
(2) Real-time PPP-B2b and Navigation Data Stream Sharing Service	25
(3) GNSS Datasets for RTKLIB-B2b & RTKLIB Playback and Post-Processing	26
a. Dataset 1: SINO & UNICORE Receiver Raw Log Data for Real-time Playback	26
b. Dataset 2: High-Rate MGEX Observations, DLR Ephemeris, and Unicore B2b Correction Data for Post-Processing	26
6. References	28
7. Appendix	30
(1) PPP-B2b Format Description	30

(2) PPP-B2b IOD consistency control with keep-last-valid update gating ...	31
(3) Observed GEO PRN Anomaly (C61/C62) and Processing Strategy	32
(4) In-house Data Acquisition Platform	33
(5) Reference Commands for Receiver Observation, Ephemeris, and PPP-B2b Data Output	34

1.Introduction

The BeiDou Global Navigation System (BDS-3) provides Precise Point Positioning (PPP) service via B2b signals, known as PPP-B2b, in China and its surrounding regions. Currently, the services provides precise satellite orbit and clock offsets for BDS and GPS constellations. Extensive researches have been conducted in evaluating the accuracies of individual products, as well as the performance in user positioning (Sun et al. 2023; Yangyin Xu, Yang, and Li 2021; Chen et al. 2025). These results suggest that the PPP-B2b service can provide accurate and robust services for users and is suitable for precise applications such as marine positioning, time transfer, and troposphere monitoring (Ying Xu, Zhao, and Wang 2024; Wang et al. 2024; Zhou et al. 2024; Geng et al. 2022).

The signal structure of PPP-B2b is identical to that of the B2b signal, both are broadcast on the I component. Upon capturing these messages, receivers internally demodulate the signal and subsequently output the decoded information as binary code to the user via the serial port or other communication interfaces (CNSO,2020). While the BDS B2b Interface Control Document (ICD) defines a standard format for this binary code, receiver manufacturers often modify it for proprietary reasons. This software is designed to decode these manufacturer-specific PPP-B2b binary correction messages and conduct PPP. It currently supports receivers from Sino (ComNavTech, SinoGNSS, also referred to as ComNav) and Unicore (Unicore Communications). Additionally, current RTCM standard has not yet incorporated BDS CNAV1 message formats, this software, for user convenience, also includes decoders for GPS LNAV and BDS CNAV1 ephemerides provided in the proprietary binary formats of SINO and UNICORE receivers.

This document is structured into the following sections to familiarize users with the functionalities of the RTKLIB-B2b software:

- (1) RTKLIB-B2b Structure, Compilation, and Installation
- (2) RTKLIB-B2b Decoder
- (3) RTKLIB-B2b Positioning
- (4) Software Maintenance and Data SharingAppendix

(5) References and Appendix

2. RTKLIB-B2b Structure, Compilation, and Installation

(1) RTKLIB-B2b Software Structure

RTKLIB-B2b is a software tool developed as an extension of the widely adopted RTKLIB package. It is specifically designed for decoding vendor-defined/proprietary receiver output messages (e.g., Unicore or ComNav formats) carrying PPP-B2b corrections and associated navigation-message information (GPS LNAV, BDS CNAV1), and for conducting PPP-B2b aided Precise Point Positioning (PPP). Note that the term “proprietary” here refers to the receiver-side output/encapsulation formats rather than the PPP-B2b definition itself.

To accommodate different application scenarios, the software provides two primary operational modes: real-time and post-processing. As shown in Fig 1, the two modes share the same decoding, caching/logging, and positioning workflow, but differ mainly in their input sources and time synchronization mechanisms.

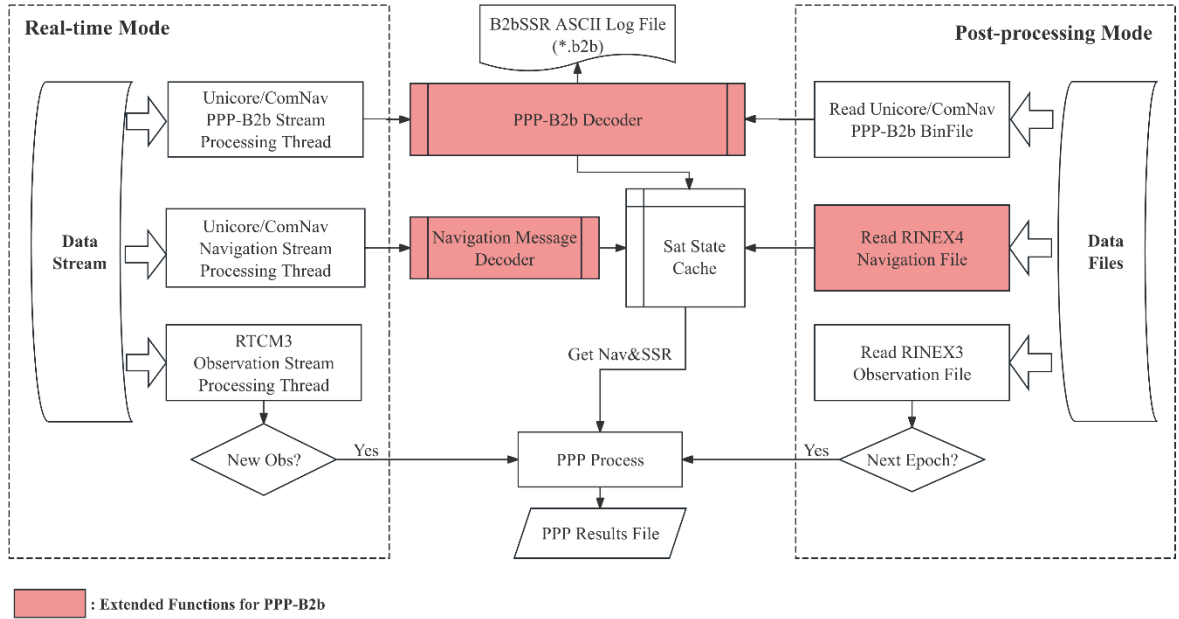


Fig 1. Flowchart of RTKLIB-B2b processing

In real-time mode, RTKLIB-B2b continuously receives live observation streams (typically in standard RTCM3 format) together with PPP-B2b corrections and navigation-related information provided through receiver output messages (Unicore/ComNav formats). The GPS LNAV navigation message itself is standardized; however, in practice, navigation-related content is often exposed by receivers through vendor-specific interfaces and message encapsulation, while BDS CNAV1 and PPP-

B2b are commonly delivered as proprietary binary streams by prevalent receivers. These real-time inputs can be accessed through standard communication interfaces such as serial ports, TCP/IP links, or via NTRIP where applicable. Internally, navigation messages and PPP-B2b corrections are decoded in their corresponding stream-processing paths and stored in caches, and the PPP computation is triggered epoch-by-epoch immediately when a new observation epoch arrives by retrieving the latest available Nav&SSR information from these caches. This epoch-driven design allows the practical transmission latency of the PPP-B2b real-time service to be reflected and evaluated in the positioning results. In addition, the exported .b2b log records both the message reference time and the receiver reception time, which can be directly used for latency analysis.

In post-processing mode, RTKLIB-B2b is intended for analyzing recorded datasets. It uses standard GNSS observation files in RINEX 3 format and merged broadcast ephemeris files in RINEX 4 format (e.g., the DLR BRD400DLR product available from the CDDIS daily GNSS data archive). PPP-B2b corrections are read from recorded binary files in Unicore/ComNav formats, which are typically logged from a receiver's real-time output using serial/network logging tools such as RTKLIB's STRSVR and then replayed for decoding and positioning. During processing, for each new observation epoch read from the RINEX observation file, the module retrieves the latest valid PPP-B2b correction set prior to that epoch and triggers the corresponding positioning update.

A key component of RTKLIB-B2b is the PPP-B2b decoder, which bridges vendor-specific receiver outputs and PPP processing by reconstructing PPP-ready correction products with integrity control. As shown in Fig 2, the decoder reads PPP-B2b binary data originating from either real-time streams or recorded binary files. For the Unicore and ComNav receivers used in this project, these binaries are vendor-specific output messages (their message formats and field definitions are described in the corresponding receiver user manuals) that are transformed internally from raw PPP-B2b broadcast messages by the receiver firmware; therefore, the decoder targets these vendor-defined output messages rather than the raw PPP-B2b broadcast frames. After identifying and parsing the relevant messages, the decoder converts the binary payloads

into correction parameters in physical units (e.g., meters for orbit/clock corrections). The proprietary aspects typically involve message framing/identifiers, field packing/ordering, and the mapping of PPP-B2b-related elements into receiver-specific output records.

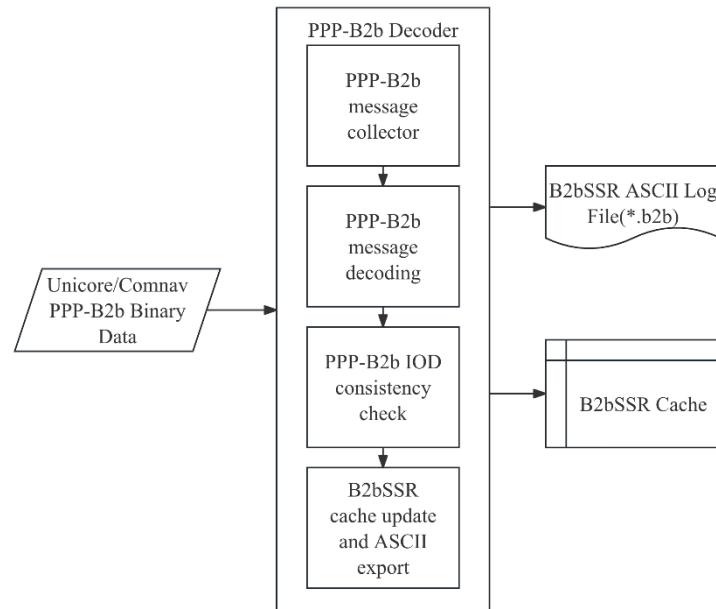


Fig 2. Flowchart of PPP-B2b Decoder

Because PPP-B2b correction components (Mask, Orbit and URAI, Clock, DCB) are broadcast asynchronously at different intervals, the decoder does not publish a PPP-ready correction set to the positioning engine upon receiving a single message. Instead, it enforces completeness and the core Issue of Data (IOD) consistency controls defined in the PPP-B2b ICD. A complete correction set is formed only when the IOD indicators are mutually consistent across related correction components and are also consistent with the currently used broadcast ephemeris. If the checks are not satisfied (e.g., data loss, or abnormal/testing signals such as those associated with specific GEO satellites), the decoder buffers the intermediate updates and incrementally refreshes the internal correction parameters until a newer complete and consistent update becomes available. A satellite-exclusion policy can also be applied to ignore selected satellites when necessary, preventing invalid corrections from contaminating the PPP solution.

Once the consistency checks are passed, the decoder packages the complete correction set into an internal B2bSSR structure. This B2bSSR structure is an internal container designed in a layout similar to RTKLIB's RTCM/IGS-SSR storage structures,

and it is the format used to provide PPP-B2b corrections to the PPP processing module. For troubleshooting, analysis, and reproducible post-processing, the decoder can additionally export the validated contents of the B2bSSR cache into a human-readable ASCII log file with the “.b2b” extension. This .b2b file follows an SSR-style ASCII log layout similar to the BNC log style for inspection, with minimal PPP-B2b-specific fields only for information that is not directly represented in standard RTCM/IGS-SSR fields. The .b2b file should therefore be interpreted as a logging output format rather than a standard RTCM/IGS-SSR product. Detailed format definition and representative output examples are provided in the later “Decoding result format description” section of this manual.

(2) RTKLIB-B2b Software Compilation and Installation

Project Repository: <https://github.com/UCAS-Liuchunbo/RTKLIB-B2b.git>

Clone the repository using the address provided above. The structure of the cloned repository is as follows:

```
.
├── CMakeLists.txt
├── src/
├── include/
├── app/
├── build/
├── lib/
├── bin/
├── conf/
├── out/
├── manual/
└── example/
```

```
le/
```

- CMakeLists.txt: The script file used by CMake to build the project.
- src/: The directory contains the common source code files.
- include/: The directory contains the common header files.
- app/: The directory contains the functional-related source code files.
- build/: The directory intended for compilation.
- lib/: The directory intended for saving common static libraries.
- bin/: The directory intended for saving the executable programs.
- conf/: The directory contains configuration files.
- out/: The directory intended for saving result files.

- manual/: The directory contains this manual file.
- example/: The directory contains sample data and configuration files. Users will primarily work within this directory to run the examples.

The functional modules of this program are compiled together with the common source code library. For different operating systems, specific compilers are required for the compilation:

- Unix: The GCC compiler is necessary.
- Windows: Use GCC from the MinGW-w64 toolchain.

Executable files are obtained directly after compilation; no additional installation steps are required. For users interested in debugging or modifying the code, we recommend using the lightweight and free IDE, VS Code.

Note: The real-time modules utilize console functionality adapted from RTKLIB. However, due to limited support for the relevant APIs in Windows, the console feature cannot be enabled for real-time modules on Windows systems (though they can still run in non-console mode). The console mode allows convenient monitoring of real-time data stream transmission and decoding status, facilitating rapid troubleshooting. Therefore, we strongly recommend using a Unix-like system (e.g., Linux, macOS) to run this software for the optimal experience.

The demonstration environment is **Ubuntu 20.04 LTS**.

First, install the necessary tools (GCC compiler suite, CMake, and GDB) using your package manager.

```
sudo apt update
sudo apt install build-essential
sudo apt install cmake
sudo apt install gdb
```

The versions used in the demonstration were:

```

book@ubuntu:/$ gcc --version
gcc (Ubuntu 9.4.0-1ubuntu1~20.04.2) 9.4.0
Copyright (C) 2019 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

book@ubuntu:/$ cmake --version
cmake version 3.16.3

CMake suite maintained and supported by Kitware (kitware.com/cmake).
book@ubuntu:/$ gdb --version
GNU gdb (Ubuntu 9.2-0ubuntu1~20.04.2) 9.2
Copyright (C) 2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
book@ubuntu:/$

```

Fig 3. Example figure (1)

Navigate into the build directory and generate the build system. (Note: Ensure the build directory is empty before running cmake. Clear all contents if necessary.)

```

cd ./build
cmake ..

book@ubuntu:~/GNSS/B2bPPP$ cd ./build/
book@ubuntu:~/GNSS/B2bPPP/build$ ls
book@ubuntu:~/GNSS/B2bPPP/build$ cmake ..
-- The C compiler identification is GNU 9.4.0
-- The CXX compiler identification is GNU 9.4.0
-- Check for working C compiler: /usr/bin/cc
-- Check for working C compiler: /usr/bin/cc -- works
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Detecting C compile features
-- Detecting C compile features - done
-- Check for working CXX compiler: /usr/bin/c++
-- Check for working CXX compiler: /usr/bin/c++ -- works
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Detecting CXX compile features
-- Detecting CXX compile features - done
-- Configuring done
-- Generating done
-- Build files have been written to: /home/book/GNSS/B2bPPP/build
book@ubuntu:~/GNSS/B2bPPP/build$

```

Fig 4. Example figure (2)

After cmake completes successfully, start the compilation:

```
make
```

```

[ 63%] Building C object src/CMakeFiles/B2bLib.dir/rcv/tersus.c.o
[ 64%] Building C object src/CMakeFiles/B2bLib.dir/rcv/ublox.c.o
[ 65%] Building C object src/CMakeFiles/B2bLib.dir/rcv/unicore.c.o
[ 67%] Building C object src/CMakeFiles/B2bLib.dir/rcv/sinan.c.o
[ 68%] Building C object src/CMakeFiles/B2bLib.dir/f2c/cal2jd.c.o
[ 69%] Building C object src/CMakeFiles/B2bLib.dir/f2c/dat.c.o
[ 71%] Building C object src/CMakeFiles/B2bLib.dir/f2c/dehanttideinel.c.o
[ 72%] Building C object src/CMakeFiles/B2bLib.dir/f2c/gmf.c.o
[ 73%] Building C object src/CMakeFiles/B2bLib.dir/f2c/gpt.c.o
[ 75%] Building C object src/CMakeFiles/B2bLib.dir/f2c/norm8.c.o
[ 76%] Building C object src/CMakeFiles/B2bLib.dir/f2c/sprod.c.o
[ 78%] Building C object src/CMakeFiles/B2bLib.dir/f2c/stidiu.c.o
[ 79%] Building C object src/CMakeFiles/B2bLib.dir/f2c/stiisem.c.o
[ 80%] Building C object src/CMakeFiles/B2bLib.dir/f2c/still.c.o
[ 82%] Building C object src/CMakeFiles/B2bLib.dir/f2c/step2diu.c.o
[ 83%] Building C object src/CMakeFiles/B2bLib.dir/f2c/step2lon.c.o
[ 84%] Building C object src/CMakeFiles/B2bLib.dir/f2c/vmf1.c.o
[ 86%] Building C object src/CMakeFiles/B2bLib.dir/f2c/vmf1ht.c.o
[ 87%] Building C object src/CMakeFiles/B2bLib.dir/f2c/zero_vec8.c.o
[ 89%] Linking C static library ../lib/libB2bLib.a
[ 89%] Built target B2bLib
Scanning dependencies of target postppp
[ 90%] Building C object app/postppp/CMakeFiles/postppp.dir/postppp.c.o
[ 91%] Linking C executable ../../bin/postppp
[ 91%] Built target postppp
Scanning dependencies of target rtppp
[ 93%] Building C object app/rtppp/CMakeFiles/rtppp.dir/rtapp.c.o
[ 94%] Linking C executable ../../bin/rtppp
[ 94%] Built target rtppp
Scanning dependencies of target postdecoder
[ 95%] Building C object app/postdecoder/CMakeFiles/postdecoder.dir/postdecoder.c.o
[ 97%] Linking C executable ../../bin/postdecoder
[ 97%] Built target postdecoder
Scanning dependencies of target rtdecoder
[ 98%] Building C object app/rtdecoder/CMakeFiles/rtdecoder.dir/rtdecoder.c.o
[100%] Linking C executable ../../bin/rtdecoder
[100%] Built target rtdecoder
book@ubuntu:~/GNSS/B2bPPP/build$

```

Fig 5. Example figure (3)

Upon successful compilation, navigate to the bin directory. You will find the compiled functional modules:

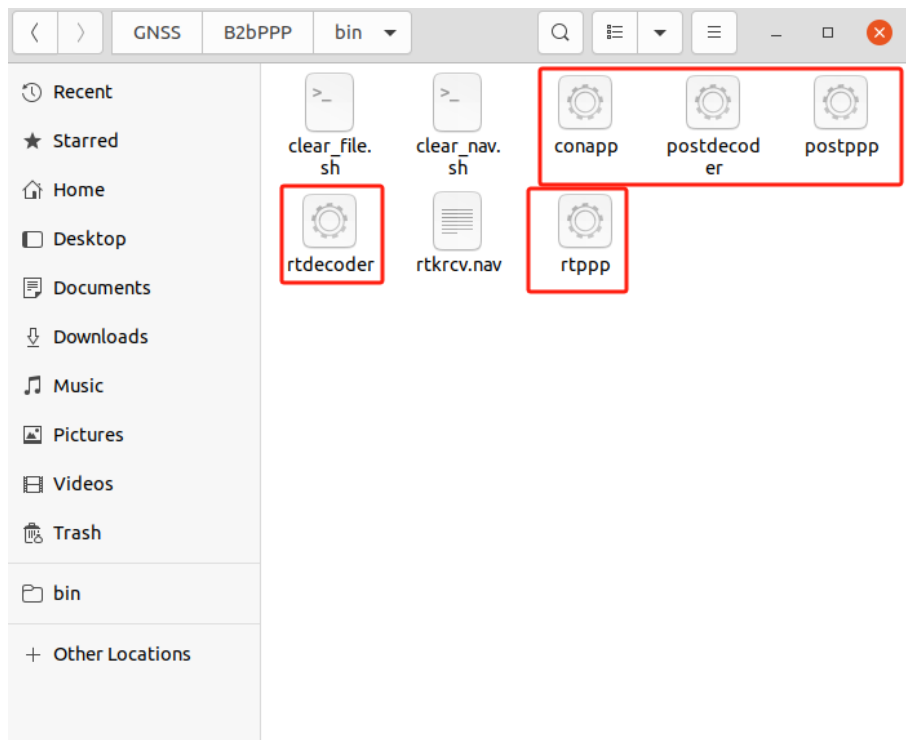


Fig 6. Example figure (4)

3. RTKLIB-B2b Decoder

Detailed specifications for the PPP-B2b formats from various manufacturers are provided in the appendix or attachments. In this section, we will introduce two different decoding methods and explain how to use the RTKLIB-B2b decoder.

(1) Post-processing Decoder

The RTKLIB-B2b post-processing decoder is designed for users who have already saved the binary data stream output from a receiver into a file, typically using serial communication software (such as RTKLIB's STRSVR). This decoder processes such files, decodes the information, and outputs it according to a specified format. Currently, this post-processing decoder supports the decoding of PPP-B2b information in both SINO and UNICORE formats.

a. Usage

The example for this module is located in the RTKLIB-B2b/example/postdecoder directory. After compilation is complete, users can replace the postdecoder program in this directory with their newly compiled version for testing. The post-processing decoder, postdecoder, is launched via command-line arguments.

```
postdecoder [-U] [-S] [-in input_B2bfile] [-out out_Resultfile]
```

b. Options

- U : Specifies decoding in UNICORE format.
- S : Specifies decoding in SINO format.
- in : Specifies the path to the input B2b Binfile.
- out : Specifies the path where the decoded B2b result file will be stored.

c. Example

```
./postdecoder -U -in ./Unicore_2024360.B2bBin -out ./UNICORE_2024360.b2b
```

```
book@ubuntu:~/GNSS/RTKLIB-B2b/example/postdecoder$ ./postdecoder -U -in ./Unicore_2024360.B2b
Decoding started...
Decoding completed!
book@ubuntu:~/GNSS/RTKLIB-B2b/example/postdecoder$
```

Fig 7. Example of running RTKLIB-B2b-PostDecoder in the command line

After decoding is complete, a text result file will be generated in the specified output directory. The format description for this file will be provided in the final part of this section.

Fig 8. Example of .b2b file obtained from decoding.

(2) Real-time Decoder

The RTKLIB-B2b real-time decoder allows users to decode the binary data stream output from a receiver in real-time via methods such as Serial, TCP/IP, Ntrip, etc., and output the results in a specified format. This real-time decoder currently supports the decoding of GPS LNAV, BDS CNAV1, and PPP-B2b information in both SINO and UNICORE formats. It also supports a playback mode (following RTKLIB playback rules).

a. Usage

The example for this module is located in the RTKLIB-B2b/test/rtdecoder directory. After compilation is complete, users can replace the rtdecoder program in this directory with their newly compiled version for testing. The real-time decoder, rtdecoder, is launched using a configuration file passed via command-line arguments. (When using a serial port for data reception, ensure the user has the necessary serial port access permissions.)

```
rtdecoder [-o config_file] [-s] [-nc]
```

b. Options

-o: Specifies the path to the configuration file.

-s: Specifies launching in console mode (this option is unavailable on Windows

systems).

-nc: Specifies launching in non-console mode.

c. Console Commands

Console commands are instructions used to interact with the program via the command line after launching in console mode (-s) to view the current real-time decoding status. The format remains consistent with RTKLIB (demo5), with modifications specific to B2b. Currently available commands are:

- sta: Prints the current status information of the program decoding, including basic software info, and counts of ephemeris and PPP-B2b messages.
- nav: Prints the currently decoded ephemeris information.
- ssr: Prints the currently decoded PPP-B2b information.
- stream: Views the current status information of the data streams.
- shutdown: Terminates the current process (when receiving data via Serial or TCP/IP, please ensure you use this command to properly terminate the process).

d. Example

Note that in this example, the configuration file is pre-filled with a test account for receiving data via Ntrip. Test accounts may experience instability. Therefore, if you have your own PPP-B2b receiver, we recommend replacing the configuration details with your own.

Enter the example directory and run the following command to start.

```
./rtdecoder -o ./rtdecoder_mycaster_sino.conf -s
book@ubuntu:~/GNSS/B2bPPP/test/rtdecoder$ ./rtdecoder -o ./rtdecoder_mycaster_sino.conf -s
** B2bPPP ver.v1.0 20250305 console (h:help) **
rtk server start
B2bPPP> sta
Parameter                : Value
B2bPPP version            : v1.0 20250305
B2bPPP server thread      : 80307968
B2bPPP server state       : run
processing cycle (ms)     : 10
# of input data stream_1  : nav(63),B2bssr(0),err(0)
# of input data stream_2  : nav(0),B2bssr(3518),err(0)
# of nav messages (sino)  : gps_lnav(1158), bds_cnav1(1100)
# of B2bssr messages (sino) : mask(48), eph_ura(191), diff_cbia(192), clk(1157), others(1885)
B2bPPP> 
```

Fig 9. Example of the printout after running the 'sta' command in the RTKLIB-B2b real-time decoder (console mode).

```

** B2bPPP ver.v1.0 20250305 console (h:help) **
rtk server start
B2bPPP> stream

Stream      Type      Fmt  S   In-byte  In-bps   Out-byte  Out-bps  Path      Message
input stream1 ntripc  sino C   1001    3968      0         0      0  @1.94.195.74: 1.94.195.74/SINO Nav
input stream2 ntripc  sino C   1178    7142      0         0      0  @1.94.195.74: 1.94.195.74/SINO_B2b
log stream1  file    -   C       0         0    1001    3970  ./Bas_%Y%m%d%H%M.log::T:
log stream2  file    -   C       0         0    1178    2517  ./Cor_%Y%m%d%H%M.log::T:
B2bPPP>

```

Fig 10. Example of the printout after running the 'stream' command in the RTKLIB-B2b real-time decoder (console mode).

```

B2bPPP server state      : Full
processing cycle (ms)    : 10
# of input data stream_1 : nav(63),B2bssr(0),err(0)
# of input data stream_2 : nav(0),B2bssr(3518),err(0)
# of nav messages (sino) : gps_lnav(1158), bds_cnav1(1100)
# of B2bssr messages (sino) : mask(48), eph_ura(191), diff_cbia(192), clk(1157), others(1885)
B2bPPP> ssr

SAT  S  UDI  IODSSR  URA      T0      D0-A    D0-C    D0-R      C0      C1      C2
G05  OK  48   1    39 2025/03/07 17:05:41 0.277 -0.640 -0.742  2.360 0.000 0.0000
G10  OK  48   1    39 2025/03/07 17:05:41 0.198 1.242 -0.173 -0.542 0.000 0.0000
G13  OK  48   1    39 2025/03/07 17:05:41 -0.061 0.858 0.307 -0.576 0.000 0.0000
G15  OK  48   1    39 2025/03/07 17:05:41 0.243 0.160 -0.691 0.376 0.000 0.0000
G18  OK  48   1    39 2025/03/07 17:05:41 0.181 -0.947 -0.186 0.000 0.000 0.0000
G23  OK  48   1    39 2025/03/07 17:05:41 0.016 0.115 -0.064 0.392 0.000 0.0000
G24  OK  48   1    39 2025/03/07 17:05:41 0.088 -1.478 0.550 -0.742 0.000 0.0000
G32  OK  48   1    39 2025/03/07 17:05:41 0.107 0.858 -0.454 -1.226 0.000 0.0000
C19  OK  48   1    31 2025/03/07 17:05:41 -0.045 0.045 -0.038 -0.029 0.000 0.0000
C20  OK  48   1    37 2025/03/07 17:01:41 0.011 0.000 -0.090 0.262 0.000 0.0000
C21  OK  48   1    31 2025/03/07 17:05:41 0.005 -0.026 0.026 0.400 0.000 0.0000
C22  OK  48   1    36 2025/03/07 17:05:41 0.021 -0.058 0.000 -0.544 0.000 0.0000
C29  OK  48   1    39 2025/03/07 16:47:17 0.027 -0.064 -0.013 -0.486 0.000 0.0000
C35  OK  48   1    31 2025/03/07 17:05:41 0.018 -0.038 -0.109 0.256 0.000 0.0000
C36  OK  48   1    31 2025/03/07 17:05:41 0.022 0.000 0.115 0.278 0.000 0.0000
C38  OK  48   1    31 2025/03/07 17:05:41 -0.027 0.083 -0.122 -0.003 0.000 0.0000
C39  OK  48   1    31 2025/03/07 17:05:41 0.018 0.032 0.531 -0.357 0.000 0.0000
C44  OK  48   1    31 2025/03/07 17:05:41 0.038 -0.038 -0.115 -0.190 0.000 0.0000
C45  OK  48   1    31 2025/03/07 17:05:41 0.005 0.013 0.077 0.840 0.000 0.0000
B2bPPP>

```

Fig 11. Example of the printout after running the 'ssr' command in the RTKLIB-B2b real-time decoder (console mode).

the original format to create the specific PPP-B2b output format for the RTKLIB-B2b decoder. The detailed format description follows.

a. Epoch Record

- The special character > marks the beginning of an "Epoch Record".
- B2bSSR Message/Topic Descriptor, Valid descriptors currently include MASK, ORBIT_URAI, DIFF_CODE_BIAS, CLOCK

These correspond respectively to PPP-B2b Information Types 1~4:

Satellite mask、Orbit corrections and User Range Accuracy (URA) index、Differential Code Bias (DCB) corrections、Clock corrections

- Message Epoch Reference Time - Year (GPS Time)
- Message Epoch Reference Time - Month (GPS Time)
- Message Epoch Reference Time - Day (GPS Time)
- Message Epoch Reference Time - Hour (GPS Time)
- Message Epoch Reference Time - Minute (GPS Time)
- Message Epoch Reference Time - Second (GPS Time)
- B2bSSR Message Update Interval (seconds):

This value might be inaccurate during initial reception and decoding. This is because, firstly, calculating this interval requires at least two messages of the same type, and secondly, it can be affected by the receiver's own startup process and the completeness of the broadcast B2b messages. In such cases, this parameter will be set to 0.

For MASK messages, if messages from multiple B2b satellites are received concurrently (e.g., PRN60 received after PRN59), the update interval field in the MASK message for PRN60 will be set to 0.

If accurate usage of this parameter is essential, it is recommended to re-iterate through the entire output file and check this value after the initial processing is complete.

- Number of Subsequent Records in this Block:

For MASK messages, this indicates the number of satellites with a mask value of 1. For other message types, this indicates the count of valid decoded information items. Note that this count of valid items is not necessarily the same as the "number of satellites" described in the interface documents, because the "number of satellites"

mentioned in interface documents might still include satellites that have been excluded due to poor accuracy. Therefore, the count here refers to the number of usable satellites whose decoded information falls within the specified accuracy threshold.

- Identifier of the Satellite Broadcasting PPP-B2b: Currently includes C59, C60, C61(62).
- Receiver Reception Time: Indicated by an @ symbol following the broadcasting satellite identifier, followed by:
 - Receiver Reception Time - Year (GPS Time)
 - Receiver Reception Time - Month (GPS Time)
 - Receiver Reception Time - Day (GPS Time)
 - Receiver Reception Time - Hour (GPS Time)
 - Receiver Reception Time - Minute (GPS Time)
 - Receiver Reception Time - Second (GPS Time)

b. Satellite Mask 'MASK' Example

```
> MASK 2025 03 07 16 59 24.0 48 59 B2bSatPrn_59 @ 2025 03 07 16 59 29.2
IOD:      1          2
BDSMASK:  0000000000000000001111111111011111111111100000000000000000
GPSMASK:  11111111111111111111111111111111110000
GALMASK:  0000000000000000000000000000000000000000000000000000
GLOMASK:  0000000000000000000000000000000000000000000000000000
BDS PRN:   C19 C20 C21 C22 C23 C24 C25 C26 C27 C28 C29 C30 C32 C33 C34 C35 C36 C37 C38 C39 C40 C41
C42 C43 C44 C45 C46
GPS PRN:   G01 G02 G03 G04 G05 G06 G07 G08 G09 G10 G11 G12 G13 G14 G15 G16 G17 G18 G19 G20 G21 G22
G23 G24 G25 G26 G27 G28 G29 G30 G31 G32
GAL PRN:
GLO PRN:
```

Records in this block provide the following satellite-specific information (by row):

- IOD: Issue of Data SSR (IODSSR) and Issue of Data PPP (IODP) respectively. IODSSR corresponds to the IODSSR of each message, while IODP corresponds to the IODP in Message Type 4 (Clock).
- BDS MASK: BDS mask represented in ASCII format.
- GPS MASK: GPS mask represented in ASCII format.
- GAL MASK: Galileo mask represented in ASCII format.
- GLO MASK: GLONASS mask represented in ASCII format.

- BDS PRN: List of all BDS satellites with MASK=1 (SLOT number converted to PRN).
- GPS PRN: List of all GPS satellites with MASK=1 (SLOT number converted to PRN).
- GAL PRN: List of all Galileo satellites with MASK=1 (SLOT number converted to PRN).
- GLO PRN: List of all GLONASS satellites with MASK=1 (SLOT number converted to PRN).

c. Orbit Corrections and URAI 'ORBIT_URAI' Example

> ORBIT_URAI 2025 03 07 16 59 17.0 0 6 B2bSatPrn_59 @ 2025 03 07 16 59 41.2										
C36	1	24	3	31	0.0224	0.0384	0.0768	0.0000	0.0000	0.0000
C37	1	24	5	31	-0.0528	-0.0896	0.0000	0.0000	0.0000	0.0000
C38	1	24	1	31	0.0064	0.0640	-0.0768	0.0000	0.0000	0.0000
C39	1	24	1	31	0.0240	-0.0832	0.7872	0.0000	0.0000	0.0000
C44	1	24	7	31	0.0032	-0.0192	-0.1088	0.0000	0.0000	0.0000
C45	1	24	1	31	0.0096	0.0512	0.0448	0.0000	0.0000	0.0000

Records in this block provide the following satellite-specific information (by column):

- Satellite PRN number (G=GPS, R=GLONASS, E=Galileo, C=BDS).
- IODSSR number for this message, corresponding to IODSSR in the MASK message.
- IODN number for this message, corresponding to IODC in the broadcast ephemeris.
- IOD Corr for this message, corresponding to IOD Corr in Message Type 4 (Clock).
- User Range Accuracy Index (URAI) for this message.
- Broadcast ephemeris orbit correction - Radial component [meters].
- Broadcast ephemeris orbit correction - Along component [meters].
- Broadcast ephemeris orbit correction - Cross component [meters].
- Broadcast ephemeris orbit correction velocity - Radial component [mm/s].
- Broadcast ephemeris orbit correction velocity - Along component [mm/s].
- Broadcast ephemeris orbit correction velocity - Along component [mm/s].

d. Differential Code Bias Corrections 'DIFF_CODE_BIAS' Example

> DIFF_CODE_BIAS 2025 03 07 16 58 38.0 0 3 B2bSatPrn_60 @ 2025 03 07 16 59 39.1											
C39	1	7	1P	2.8900	7I -1.5810	7Q -1.0880	2I 1.9720	1D 2.6860	5D -2.7880	5P -1.9890	
C44	1	7	1P	-0.6460	7I -1.4450	7Q -0.9180	2I -0.3570	1D -0.8500	5D -2.0910	5P -1.3260	
C45	1	7	1P	7.8200	7I -2.3120	7Q -1.8190	2I 5.7120	1D 7.5650	5D -3.2640	5P -2.4990	

Records in this block provide the following satellite-specific information (by column):

- Satellite PRN number (G=GPS, R=GLONASS, E=Galileo, C=BDS).
- IODSSR number for this message, corresponding to IODSSR in the MASK message.
- Signal and tracking mode indicators (e.g., 1C, 5Q, indicating frequency band and modulation).
- Differential Code Bias (DCB) value for the corresponding signal [meters], arranged in "Indicator-BiasValue" pairs.

e. Clock Corrections 'CLOCK' Example

> CLOCK 2025 03 07 17 09 24.0 6 9 B2bSatPrn_60 @ 2025 03 07 17 09 30.1						
C19	1	2	-0.0352	0.0000	0.0000	
C21	1	2	0.4064	0.0000	0.0000	
C22	1	2	-0.5616	0.0000	0.0000	
C34	1	2	0.2752	0.0000	0.0000	
C35	1	2	0.2592	0.0000	0.0000	
C36	1	2	0.2752	0.0000	0.0000	
C37	1	2	-0.1328	0.0000	0.0000	
C38	1	2	-0.0032	0.0000	0.0000	
C39	1	2	-0.3728	0.0000	0.0000	

Records in this block provide the following satellite-specific information (by column):

- Satellite PRN number (G=GPS, R=GLONASS, E=Galileo, C=BDS).
- IOD SSR
- IODP
- Clock correction polynomial coefficient C0 [meters].
- Clock correction polynomial coefficient C1 [mm/s].
- Clock correction polynomial coefficient C2 [mm/s²].

4. RTKLIB-B2b Positioning

(1) RTKLIB-B2b Post-processing PPP Positioning

The RTKLIB-B2b post-processing positioning functional module (Abbreviated as

RTKLIB-B2b PostPPP) is an extension built upon the RTKLIB-B2b post-processing decoder and RTKLIB's rnx2rtkp module. By using post-saved binary PPP-B2b files combined with broadcast ephemeris in RINEX 4 format (Montenbruck & Steigenberger, 2022), users can perform PPP positioning for observation stations located within the PPP-B2b coverage area.

a. Usage

The example for this module is located in the RTKLIB-B2b/example/postppp directory. After compilation is complete, users can replace the postppp program in this directory with their newly compiled version for testing. The post-processing positioning module, postppp, is launched using a configuration file passed via command-line arguments. To facilitate batch processing, it also allows users to override the input file, output file, satellite systems for processing, and processing mode via command-line parameters.

```
postppp [-o config_file] [-in input_file] [-out output_file] [-s system] [-m mode]
```

b. Options

- o: Specifies the path to the configuration file.
- in: Passes the input file path, overriding the input file specified in the configuration file (optional).
- out: Specifies the output file path, overriding the output file specified in the configuration file (optional).
- s: Specifies the satellite system(s) to be included in the solution, overriding the system(s) specified in the configuration file (optional).
- m: Specifies the processing mode, overriding the mode specified in the configuration file (optional).

c. Example

The data required for this post-processing example is publicly available from the Zenodo archive at: <https://doi.org/10.5281/zenodo.15110885>

This example demonstrates performing RTKLIB-B2b post-processing positioning for stations WUH2, GAMG, and ULAB using data from DOY 065,066 2025. Navigate to the example directory (typically RTKLIB-B2b/example/postppp) in your terminal and execute the following commands:

```
./postppp -o ./conf/postppp_25065_wuh2.conf
```

```

./postppp -o ./conf/postppp_25065_gamg.conf
./postppp -o ./conf/postppp_25065_ulab.conf
./postppp -o ./conf/postppp_25066_wuh2.conf
./postppp -o ./conf/postppp_25066_gamg.conf
./postppp -o ./conf/postppp_25066_ulab.conf

```

During this process, the command-line terminal should display warning messages related to PPP-B2b matching and information about the processing progress.

```

book@ubuntu: ~/GNSS/B2bPPP/example/postppp
book@ubuntu: ~/GN...  book@ubuntu: ~/GN...  book@ubuntu: ~/GNS...
warning: C40 eph(7) and clk(0) iodicorr is different!
warning: C41 eph(5) and clk(6) iodicorr is different!
warning: C44 eph(3) and clk(4) iodicorr is different!
warning: C35 eph(3) and clk(4) iodicorr is different!
warning: C38 eph(7) and clk(0) iodicorr is different!
warning: C39 eph(6) and clk(7) iodicorr is different!
warning: C40 eph(7) and clk(0) iodicorr is different!
warning: C41 eph(5) and clk(6) iodicorr is different!
warning: C03 eph(5) and clk(6) iodicorr is different!
warning: C04 eph(4) and clk(5) iodicorr is different!
warning: C09 eph(1) and clk(2) iodicorr is different!
warning: C16 eph(4) and clk(5) iodicorr is different!
warning: C44 eph(3) and clk(4) iodicorr is different!
warning: C26 eph(5) and clk(6) iodicorr is different!
warning: C27 eph(2) and clk(3) iodicorr is different!
warning: C28 eph(0) and clk(1) iodicorr is different!
warning: C31 eph(1) and clk(2) iodicorr is different!
warning: C32 eph(3) and clk(4) iodicorr is different!
warning: C42 eph(1) and clk(2) iodicorr is different!
warning: C42 eph(1) and clk(2) iodicorr is different!
warning: C42 eph(1) and clk(2) iodicorr is different!
warning: C42 eph(1) and clk(2) iodicorr is different!
warning: C42 eph(1) and clk(2) iodicorr is different!
processing : 2025/03/07 00:10:10 Q=6

```

Fig 14. RTKLIB-B2b PostPPP Execution Example.

After processing is complete, users should obtain three files for each station: a file ending in .pos containing the positioning results, a file ending in .stat containing the processing status information, and a .pos file containing _events in its name, which stores results from epochs judged as abnormal during processing (under normal circumstances, this file should not contain any positioning results).

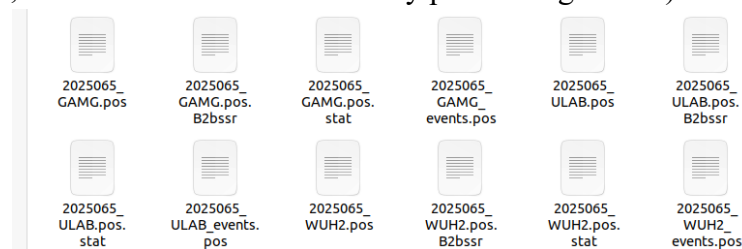


Fig 15. Generated Output Files (DOY 065)

Users can directly plot the positioning results using RTKLIB's rtkplot (The true value coordinates are derived from the IGS weekly SNX file of the corresponding week):

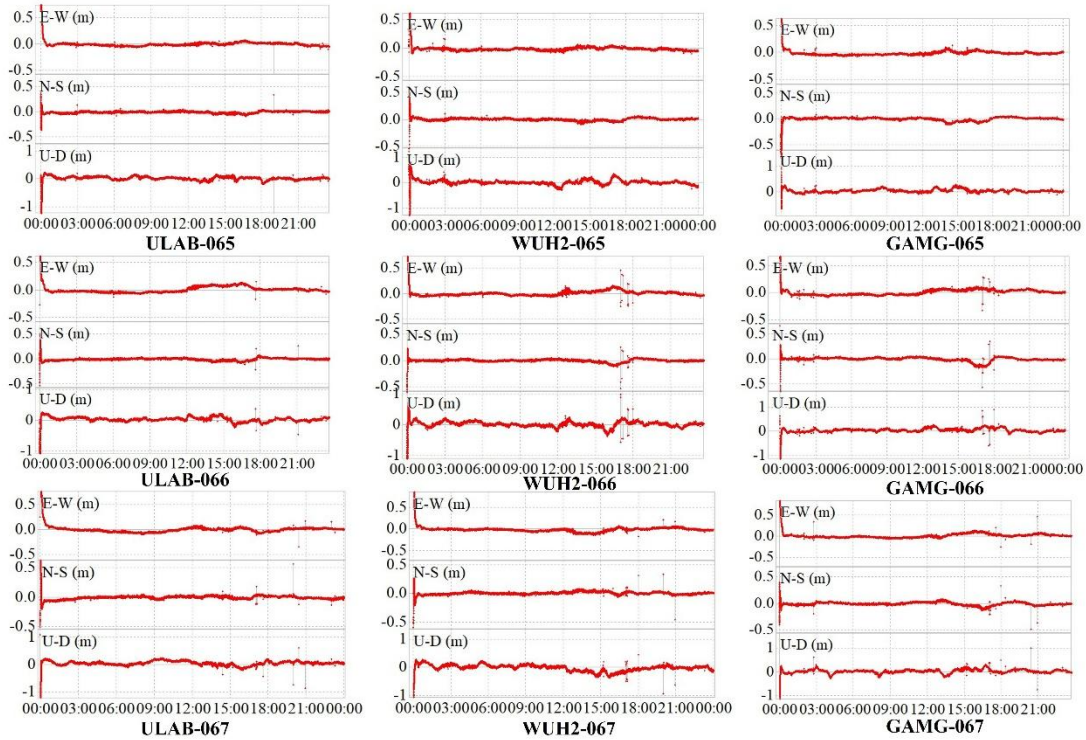


Fig 16. Example results Plotted using RTKLIB's rtkplot.

(2) RTKLIB-B2b Real-time PPP Positioning

The RTKLIB-B2b real-time positioning functional module represents an extension built upon the RTKLIB-B2b real-time decoder and incorporates functionalities adapted from RTKLIB's rtkrcv module. Utilizing real-time data streams containing PPP-B2b corrections and broadcast ephemerides, users can perform real-time Precise Point Positioning (PPP) for observation stations situated within the PPP-B2b service coverage area.

a. Usage

Example usage scenarios and files for this module are located in the RTKLIB-B2b/ example/rtppp directory. After successful compilation of the software suite, users should ensure the compiled rtppp executable is present in this directory (potentially replacing any placeholder or older version) before conducting tests.

```
rtppp [-o config_file] [-s] [-nc]
```

b. Options

- o: Specifies the full path to the configuration file.
- s: Launches the program in console mode, allowing interactive commands. (This option is unavailable on Windows systems).
- nc: Launches the program in non-console (background) mode.

-t: Specifies the debug trace level (typically written to the .trace file). If debugging is not required, a setting of 0 (off) is recommended, as trace output files can often become very large.

-r: Specifies the level of detail for the solution status output (typically written to the .stat file). A setting of 2 is recommended, as analyzing the information output at this level often helps diagnose most processing problems or issues.

c. Console Commands

When operating in console mode (-s), users can interact with the running rtppp process using specific commands entered at the prompt. These commands allow monitoring of the real-time processing status. The command structure maintains consistency with RTKLIB (demo5), incorporating further modifications tailored for PPP-B2b functionality.

The currently available console commands are:

sta : Prints current status information, including basic software version details, the number of valid ephemeris entries, and the count of received/processed PPP-B2b messages.

obs : Prints information about the currently processed observations.

nav : Prints information about the currently stored broadcast ephemeris data.

ssr : Prints information about the currently decoded PPP-B2b correction data.

stream : Displays the status of all active input/output data streams (e.g., connection status, byte counts).

shutdown: Terminates the rtppp process cleanly. Note: When data streams are configured using serial ports or TCP/IP connections, it is imperative to use this command to properly close connections and terminate the process, rather than abruptly closing the console window.

d. Example

Considering the nature of real-time data and the need for repeatable testing, an archived dataset specifically prepared for use with the RTKLIB-B2b software's playback mode is provided. (Details on RTKLIB's playback mode principles can be found in the RTKLIB documentation). This dataset is publicly available for download from the Zenodo archive at: <https://doi.org/10.5281/zenodo.15105929>

```
./rtppp -o ./conf/rtppp_25080_unicore.conf -s -t 0 -r 2
```

```
./rtppp -o ./conf/rtppp_25081_unicore.conf -s -t 0 -r 2
./rtppp -o ./conf/rtppp_25082_unicore.conf -s -t 0 -r 2
```

During console operation, users can monitor the solution status and related information by inputting commands at the prompt. The three figures below illustrate examples of the information output to the console upon executing the ‘sta’, ‘nav’, and ‘ssr’ commands.

```
book@ubuntu: ~/GNSS/B2bPPP/example/rtppp
pos xyz float std (m) rover : 0.476,0.820,0.581
ant type rover : CNTAT360
ant delta rover : 0.000 0.000 0.000
rtkrvc> sta

Parameter : Value
B2bPPP version : v1.0 20250305
B2bPPP server thread : -992540928
B2bPPP server state : run
processing cycle (ms) : 10
positioning mode : PPP-kinema
satpositioning ephopt: : brdc+b2b
frequencies : L1+L2
accumulated time to run : 00:00:16.7
cpu time for a cycle (ms) : 2
missing obs data count : 0
# of input data rover : obs(83),nav(0),gnav(0),ion(0),sbs(0),pos(0),dgps(0),ssr(0),B2bssr(0),err(0)
# of input data base : obs(0),nav(61),gnav(0),ion(0),sbs(0),pos(0),dgps(0),ssr(0),B2bssr(0),err(0)
# of input data corr : obs(0),nav(0),gnav(0),ion(0),sbs(0),pos(0),dgps(0),ssr(0),B2bssr(0),err(0)
# of rover messages (rtens) : 1074(82),1084(83),1094(83),1124(249)
# of base messages (unicore): gps_lnav(360), bds_cnav1(837)
# of corr messages (unicore): mask(2), eph_ura(16), dlff_cbia(16), clk(84), others(0)
solution status : PPP-FLOAT
time of receiver clock rover: 2025/03/21 00:00:53.000000004
time sys offset (ns) : 10608361913.163,10608361913.163,10608361913.163,0.000
solution interval (s) : 1.000
# of satellites rover : 36
# of satellites base : 0
# of valid satellites : 21
GDOP/PDOP/HDOP/VDOP : 1.1,1.0,0.6,0.8
# of real estimated states : 218
# of all estimated states : 218
pos xyz single (m) rover : -2267046.019,5006692.098,3225622.085
pos xyz float (m) rover : -2267046.019,5006692.098,3225622.085
pos xyz float std (m) rover : 0.376,0.645,0.453
ant type rover : CNTAT360
ant delta rover : 0.000 0.000 0.000
rtkrvc>
```

Fig 17. Example of the printout after running the 'sta' command in the RTKLIB-B2b real-time PPP (console mode).

```
rtkrvc> nav

SAT S IOD IOC FRQ A/A SVH Toe Toc Ttr/Tof L2C L2P
G01 - 125 125 0 4 000 2025/03/20 22:00:00 2025/03/20 22:00:00 2025/03/20 23:59:35 0 0
G02 - 85 85 0 2 000 2025/03/20 20:00:00 2025/03/20 20:00:00 2025/03/20 23:59:35 0 0
G03 OK 14 270 0 4 000 2025/03/21 02:00:00 2025/03/21 02:00:00 2025/03/21 00:00:20 0 0
G04 OK 67 835 0 2 000 2025/03/21 02:00:00 2025/03/21 02:00:00 2025/03/21 00:00:20 0 0
G05 - 67 67 0 2 000 2025/03/20 18:00:00 2025/03/20 18:00:00 2025/03/20 23:59:35 0 0
G07 OK 67 67 0 4 000 2025/03/21 02:00:00 2025/03/21 02:00:00 2025/03/21 00:00:40 0 0
G08 OK 30 30 0 2 000 2025/03/21 02:00:00 2025/03/21 02:00:00 2025/03/21 00:00:20 0 0
G09 OK 47 47 0 2 000 2025/03/21 02:00:00 2025/03/21 02:00:00 2025/03/21 00:00:20 0 0
G10 - 61 61 0 2 000 2025/03/20 22:00:00 2025/03/20 22:00:00 2025/03/20 23:59:30 0 0
G12 - 54 54 0 2 000 2025/03/20 22:00:00 2025/03/20 22:00:00 2025/03/20 23:59:30 0 0
G13 - 78 78 0 2 000 2025/03/20 18:00:00 2025/03/20 18:00:00 2025/03/20 23:59:30 0 0
G15 - 52 52 0 2 000 2025/03/20 20:00:00 2025/03/20 20:00:00 2025/03/20 23:59:30 0 0
G16 OK 83 83 0 2 000 2025/03/21 02:00:00 2025/03/21 02:00:00 2025/03/21 00:00:20 0 0
G18 OK 203 715 0 2 000 2025/03/21 02:00:00 2025/03/21 02:00:00 2025/03/21 00:00:20 0 0
G21 - 15 15 0 2 03F 2025/03/21 02:00:00 2025/03/21 02:00:00 2025/03/21 00:00:10 0 0
G23 - 209 977 0 2 000 2025/03/20 22:00:00 2025/03/20 22:00:00 2025/03/20 23:59:30 0 0
G24 - 68 68 0 2 000 2025/03/20 20:00:00 2025/03/20 20:00:00 2025/03/20 23:59:30 0 0
G25 - 11 11 0 2 000 2025/03/20 22:00:00 2025/03/20 22:00:00 2025/03/20 23:59:30 0 0
G26 OK 187 187 0 4 000 2025/03/21 02:00:00 2025/03/21 02:00:00 2025/03/21 00:00:20 0 0
G27 OK 83 83 0 4 000 2025/03/21 02:00:00 2025/03/21 02:00:00 2025/03/21 00:00:20 0 0
G28 OK 13 781 0 2 000 2025/03/21 02:00:00 2025/03/21 02:00:00 2025/03/21 00:00:20 0 0
G29 OK 126 126 0 2 000 2025/03/21 00:00:00 2025/03/21 00:00:00 2025/03/20 23:59:30 0 0
G31 OK 40 40 0 2 000 2025/03/21 02:00:00 2025/03/21 02:00:00 2025/03/21 00:00:20 0 0
G32 OK 30 30 0 2 000 2025/03/21 00:00:00 2025/03/21 00:00:00 2025/03/20 23:59:30 0 0
C21 - 5 5 0 0 000 2025/03/20 21:00:14 2025/03/20 21:00:14 2025/03/20 23:59:30 0 0
C22 - 2 2 0 0 000 2025/03/20 18:00:14 2025/03/20 18:00:14 2025/03/20 23:59:30 0 0
C23 OK 8 8 0 0 000 2025/03/21 00:00:14 2025/03/21 00:00:14 2025/03/21 00:00:35 0 0
C24 OK 8 8 0 0 000 2025/03/21 00:00:14 2025/03/21 00:00:14 2025/03/21 00:00:35 0 0
C25 OK 8 8 0 0 000 2025/03/21 00:00:14 2025/03/21 00:00:14 2025/03/21 00:00:35 0 0
C26 - 6 6 0 0 000 2025/03/20 22:00:14 2025/03/20 22:00:14 2025/03/20 23:59:30 0 0
C32 OK 8 8 0 0 000 2025/03/21 00:00:14 2025/03/21 00:00:14 2025/03/21 00:00:35 0 0
C33 OK 8 8 0 0 000 2025/03/21 00:00:14 2025/03/21 00:00:14 2025/03/21 00:00:35 0 0
C34 OK 8 8 0 0 000 2025/03/21 00:00:14 2025/03/21 00:00:14 2025/03/21 00:00:35 0 0
C38 OK 7 7 0 0 000 2025/03/20 23:00:14 2025/03/20 23:00:14 2025/03/20 23:59:30 0 0
C39 OK 8 8 0 0 000 2025/03/21 00:00:14 2025/03/21 00:00:14 2025/03/21 00:00:35 0 0
C40 OK 8 8 0 0 000 2025/03/21 00:00:14 2025/03/21 00:00:14 2025/03/21 00:00:35 0 0
C41 OK 8 8 0 0 000 2025/03/21 00:00:14 2025/03/21 00:00:14 2025/03/21 00:00:35 0 0
C42 OK 7 7 0 0 000 2025/03/20 23:00:14 2025/03/20 23:00:14 2025/03/20 23:59:30 0 0
C43 OK 8 8 0 0 000 2025/03/21 00:00:14 2025/03/21 00:00:14 2025/03/21 00:00:35 0 0
C44 OK 8 8 0 0 000 2025/03/21 00:00:14 2025/03/21 00:00:14 2025/03/21 00:00:35 0 0
C45 - 3 3 0 0 000 2025/03/20 19:00:14 2025/03/20 19:00:14 2025/03/20 23:59:30 0 0
ION: 0.00E+00 0.00E+00 0.00E+00 0.00E+00 0.00E+00 0.00E+00 0.00E+00 0.00E+00
UTC: 0.00E+00 0.00E+00 0.00E+00 0.00E+00 0.00E+00 0.00E+00 0.00E+00 0.00E+00
```

Fig 18. Example of the printout after running the 'nav' command in the RTKLIB-B2b real-time PPP (console mode).

```

solution interval (s)      : 1.000
# of satellites rover      : 37
# of satellites base       : 0
# of valid satellites      : 19
GDOP/PDOP/HDOP/VDOP      : 1.3,1.1,0.6,1.0
# of real estimated states : 218
# of all estimated states  : 218
pos xyz single (m) rover  : -2267045,249,5006691.596,3225621.816
pos xyz float (m) rover   : -2267045,249,5006691.596,3225621.816
pos xyz float std (m) rover : 0.030,0.036,0.022
ant type rover            : CNTAT360 NONE
ant delta rover           : 0.000 0.000 0.000
rtkrcv> ssr

SAT  S  UDI  IODSSR  URA      T0      D0-A  D0-C  D0-R      C0      C1      C2
G03 OK 48 1 39 2025/03/21 00:13:41 0.019 -0.141 -0.102 -2.987 0.000 0.0000
G04 OK 48 1 39 2025/03/21 00:13:41 0.144 -1.011 -0.365 0.125 0.000 0.0000
G07 OK 48 1 39 2025/03/21 00:13:41 0.282 0.038 0.525 0.000 0.000 0.0000
G08 OK 48 1 39 2025/03/21 00:13:41 0.064 -0.410 0.045 -0.109 0.000 0.0000
G09 OK 48 1 39 2025/03/21 00:13:41 0.050 0.006 0.448 -0.658 0.000 0.0000
G16 OK 48 1 39 2025/03/21 00:13:41 -0.029 1.126 0.102 1.331 0.000 0.0000
G18 OK 48 1 37 2025/03/21 00:13:41 -0.126 -0.416 1.133 -1.230 0.000 0.0000
G26 OK 48 1 39 2025/03/21 00:13:41 -0.237 1.274 0.371 0.477 0.000 0.0000
G27 OK 48 1 31 2025/03/21 00:13:41 -0.250 0.154 -0.147 -0.763 0.000 0.0000
G28 OK 48 1 39 2025/03/21 00:13:41 -0.174 -0.326 -0.019 0.491 0.000 0.0000
G31 OK 48 1 39 2025/03/21 00:13:41 0.093 -1.747 -0.832 0.078 0.000 0.0000
C23 OK 48 1 31 2025/03/21 00:13:41 0.005 -0.077 0.090 -0.274 0.000 0.0000
C24 OK 48 1 31 2025/03/21 00:13:41 -0.046 -0.058 -0.090 -0.002 0.000 0.0000
C25 OK 48 1 31 2025/03/21 00:13:41 -0.008 -0.134 -0.026 -0.552 0.000 0.0000
C32 OK 48 1 31 2025/03/21 00:13:41 0.011 -0.077 -0.122 0.479 0.000 0.0000
C33 OK 48 1 31 2025/03/21 00:13:41 -0.010 -0.051 -0.122 0.168 0.000 0.0000
C34 OK 48 1 31 2025/03/21 00:13:41 -0.037 0.026 0.003 -0.443 0.000 0.0000
C39 OK 48 1 31 2025/03/21 00:13:41 -0.005 0.000 0.160 -0.616 0.000 0.0000
C40 OK 48 1 23 2025/03/21 00:13:41 -0.002 0.102 0.147 0.467 0.000 0.0000
C41 OK 48 1 31 2025/03/21 00:13:41 -0.011 0.038 -0.128 -0.274 0.000 0.0000
C43 OK 48 1 31 2025/03/21 00:13:41 -0.016 0.000 0.128 0.306 0.000 0.0000
C44 OK 48 1 31 2025/03/21 00:13:41 -0.046 -0.019 0.058 -0.402 0.000 0.0000
rtkrcv>

```

Fig 19. Example of the printout after running the 'ssr' command in the RTKLIB-B2b real-time PPP (console mode).

The resulting positioning solutions stored in the .pos file can be directly plotted and analyzed using RTKLIB's rtkplot tool. For validation purposes, reference coordinates were established using final precise orbit and clock products published by the European Space Agency (ESA). These coordinates represent the average values obtained from converged static PPP solutions. In the Earth-Centered Earth-Fixed (ECEF) frame, these reference coordinates are : X = -2267045.334 m Y = 5006691.635 m Z = 3225621.891 m

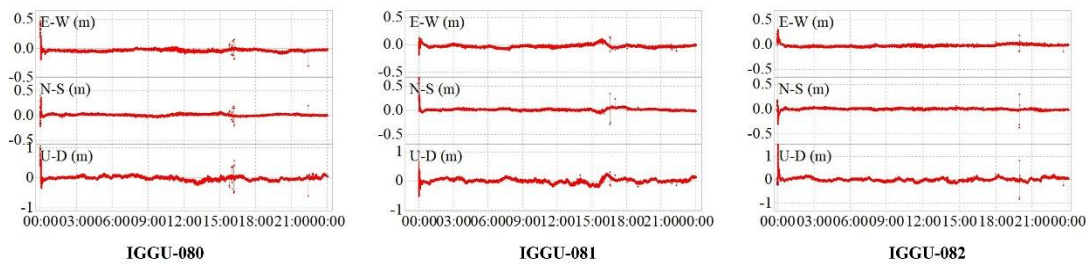


Fig 20. Example results Plotted using RTKLIB's rtkplot.

5. Software Maintenance and Data Sharing

(1) Software Maintenance

With the ongoing development of the BeiDou Navigation Satellite System, PPP-B2b services are expected to support more satellite systems and correction types in the future. We will continue to maintain this software on GitHub. We sincerely welcome users to report any issues encountered during usage by creating an issue. To facilitate a quicker resolution, we recommend using the following template:

```
## Issue Type
[ ] Data Parsing Anomaly
[ ] Positioning Result Deviation
[ ] Other Issues

## Conditions to Reproduce
1. Receiver Model: _____
2. Data File: _____ (If using an example provided with the software, just provide the filename. For other data, please provide a secure download link)
3. Operating Environment: _____

## Key Log Information
(Paste/upload snippets of log messages or errors you believe are relevant. If using an IDE, we currently only accept error reports related to VS Code.)

## Expected Behavior
(Clearly describe the expected outcome or result)

## Additional Information
(Screenshots, etc. Format is flexible)
```

(2) Real-time PPP-B2b and Navigation Data Stream Sharing Service

Considering that some users may not have convenient long-term access to a PPP-B2b receiver for data collection, we provide a sharing service for real-time PPP-B2b and ephemeris data streams. Our antenna is located on the rooftop of a building within the campus of the Innovation Academy for Precision Measurement Science and Technology (APM) in Wuhan, China. Data is received using a SINO_K803W receiver and a CNT AT360 antenna, and broadcast externally via an NtripCaster.

The free test account details are provided below. Should there be any changes, we will update this section of the documentation on GitHub.

```
Address: 1.94.195.74
Port: 2101
Test Account Username: user2
Test Account Password: pass2
PPP-B2b Information Mount Point: SINO_B2b
Ephemeris Information Mount Point: SINO_Nav
```

Please note: This account is provided solely for the purpose of testing PPP-B2b and ephemeris information decoding. It must not be

used for any other purpose. While we will strive to maintain its stability, we reserve the right to deactivate this account at any time without prior notice to users.

(3) GNSS Datasets for RTKLIB-B2b & RTKLIB Playback and Post-Processing

This public archived dataset collection contains two complementary GNSS datasets designed to support research, algorithm development, and software testing/validation (e.g., for RTKLIB-B2b, RTKLIB) involving Precise Point Positioning (PPP) utilizing BeiDou PPP-B2b corrections. These two datasets cater specifically to two typical application scenarios: real-time playback simulation and post-processing analysis.

a. Dataset 1: SINO & UNICORE Receiver Raw Log Data for Real-time Playback

To provide binary format data, as output directly by receivers, required for simulating real-time data streams; suitable for testing the real-time processing (playback functionality) of software such as RTKLIB-B2b and RTKLIB.

Data was collected on the rooftop of Building 1 at the Innovation Academy for Precision Measurement Science and Technology (APM), CAS, Wuhan, China, using two types of GNSS receivers, ComNav K803W and Unicore Communications (UNICORE) UM980, along with a CNT AT360 antenna.

Time Period Covered: Day of Year (DOY) 080 to 082, 2025 (corresponding to March 21-23, 2025).

It is now available for users to download in Zenodo:

<https://doi.org/10.5281/zenodo.15105929>

b. Dataset 2: High-Rate MGEX Observations, DLR Ephemeris, and Unicore B2b Correction Data for Post-Processing

To provide data in standard or common formats suitable for PPP post-processing analysis, with a particular focus on evaluating the performance and application effectiveness of PPP-B2b correction services.

Observation data originates from three IGS MGEX stations: gamg, ulab, and wuh2. Broadcast ephemeris is sourced from the German Aerospace Center (DLR). PPP-B2b correction data comes from independent logging by a Unicore receiver. Observation data covers DOY 065-067, 2025. Broadcast ephemeris and PPP-B2b correction data

cover DOY 065-068, 2025 (corresponding to March 6-9, 2025).

It is now available for users to download in Zenodo:

<https://doi.org/10.5281/zenodo.15110885>

6. References

1. China Satellite Navigation Office. BeiDou Navigation Satellite System Signal In Space Interface Control Document Precise Point Positioning Service Signal PPP-B2b (Version 1.0) 2020. Available online: <http://www.beidou.gov.cn/xt/gfxz/202008/P020200803362062482940.pdf>.
2. “BKG Ntrip Client - BNC.” n.d. Accessed March 31, 2025. <https://igs.bkg.bund.de/ntrip/bnc>.
3. Montenbruck, O., & Steigenberger, P. (2022). BRD400DLR: DLR’s merged multi-GNSS broadcast ephemeris product in RINEX 4 format [Dataset]. DLR/GSOC. <https://doi.org/10.57677/BRD400DLR>
4. Chen, Junping, Ziyuan Song, Yize Zhang, Junsheng Ding, Xin Jiang, Sainan Yang, and Yangfei Hou. 2025. “Clock Systematic Jump Estimation and URA Refinement of BDS-3 B2b Real-Time Precise Point Positioning Service.” *GPS Solutions* 29 (2): 73. <https://doi.org/10.1007/s10291-025-01835-7>.
5. Geng, Tao, Zhiqiang Li, Xin Xie, Wenjian Liu, Yixuan Li, and Qile Zhao. 2022. “Real-Time Ocean Precise Point Positioning with BDS-3 Service Signal PPP-B2b.” *Measurement* 203 (November):111911. <https://doi.org/10.1016/j.measurement.2022.111911>.
6. Motooka, Norizumi, Rui Hirokawa, Kenji Nakakuki, Seigo Fujita, Masakazu Miya, and Yuki Sato. 2019. “CLASLIB: An Open-Source Toolkit for Low-Cost High-Precision PPP-RTK Positioning.” 3695–3707. <https://doi.org/10.33012/2019.16977>.
7. Ren, Zhiling, Hang Gong, Jing Peng, Chengpan Tang, Xinming Huang, and Guangfu Sun. 2021. “Performance Assessment of Real-Time Precise Point Positioning Using BDS PPP-B2b Service Signal.” *Advances in Space Research* 68 (8): 3242–54. <https://doi.org/10.1016/j.asr.2021.06.006>.
8. Sun, Shuang, Min Wang, Changjian Liu, Xin Meng, and Rui Ji. 2023. “Long-Term Performance Analysis of BDS-3 Precise Point Positioning (PPP-B2b) Service.” *GPS Solutions* 27 (2): 69. <https://doi.org/10.1007/s10291-023-01409-5>.
9. Takasu, Tomoji, and Akio Yasuda. n.d. “Development of the Low-Cost RTK-GPS Receiver with an Open Source Program Package RTKLIB.”
10. Wang, Xiaoming, Yufei Chen, Jinglei Zhang, Cong Qiu, Kai Zhou, Haobo Li, and Qiuying Huang. 2024. “Assessment of BDS-3 PPP-B2b Service and Its Applications for the Determination of Precipitable Water Vapour.” *Atmosphere* 15 (9): 1048. <https://doi.org/10.3390/atmos15091048>.
11. Xu, Yangyin, Yuanxi Yang, and Jinlong Li. 2021. “Performance Evaluation of BDS-3 PPP-B2b Precise Point Positioning Service.” *GPS Solutions* 25 (4): 142. <https://doi.org/10.1007/s10291-021-01175-2>.
12. Xu, Ying, Panpan Zhao, and Jin Wang. 2024. “Precipitable Water Vapor Retrieval for Rainfall Forecasting Using BDS-3 PPP-B2b Signal in the Coastal Region of China.” *Measurement Science and Technology* 35 (11): 116309. <https://doi.org/10.1088/1361-6501/ad6471>.
13. Zhang, Runzhi, Rui Tu, and Xiaochun Lu. 2024. “HASPPP: An Open-Source Galileo HAS Embeddable RTKLIB Decoding Package.” *GPS Solutions* 28 (4):

169. <https://doi.org/10.1007/s10291-024-01706-7>.
14. Zhou, Peiyuan, Zhongkai Zhang, Zejun Liu, Daqian Lyu, Guorui Xiao, Kai Xiao, and Lan Du. 2024. “Real-Time Precise Zenith Tropospheric Delay Estimation With BDS PPP-B2b, Galileo HAS, and QZSS MADOCA-PPP Services.” *IEEE Transactions on Geoscience and Remote Sensing* 62:1–11. <https://doi.org/10.1109/TGRS.2024.3443884>.

7. Appendix

(1) PPP-B2b Format Description

The standard format specification for PPP-B2b messages has been detailed comprehensively in the official documentation. In this section, we focus solely on describing the message organization employed in the custom PPP-B2b formats implemented by two specific vendors: Sino and Unicore.

a. SINO

The complete data manual is shown in detail:

<https://www.sinognss.com/service/download/>

The PPP-B2b message structure utilized by Sino receivers is fundamentally based on the standard format, but introduces modifications primarily within the message header information. A diagram illustrating the correspondence between the standard format organization and the Sino organization is provided below.

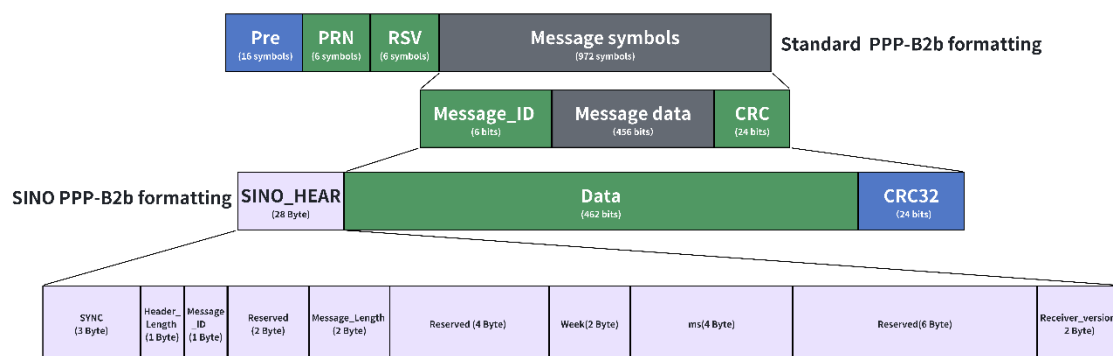


Fig Mapping between the standard PPP-B2b message framing and the ComNav receiver

output format.

b. UNICORE

The complete data manual is shown in detail:

<https://www.unicorecomm.com/download>

The organization of PPP-B2b messages from Unicore receivers differs more significantly from the standard format. Generally, in the standard specification, the sequence composed of the Preamble, PRN (Pseudo-Random Noise code identifying the satellite transmission source), and Reserved bits (Pre+PRN+RSV) is considered the message header, while the subsequent "Message Symbols" constitute the message data payload. In the Unicore PPP-B2b implementation, the PRN number (identifying the

transmitting satellite) is relocated from the header section into the message data payload. Conversely, the Message Type identifier (Message ID) is stored within the header section. The relationship between these two structures is depicted in the figure below. However, it is important to note that even within the message data payload section, certain differences in the sequential order of specific data fields may still exist compared to the standard. Users should consult the official Unicare message format documentation for precise details.

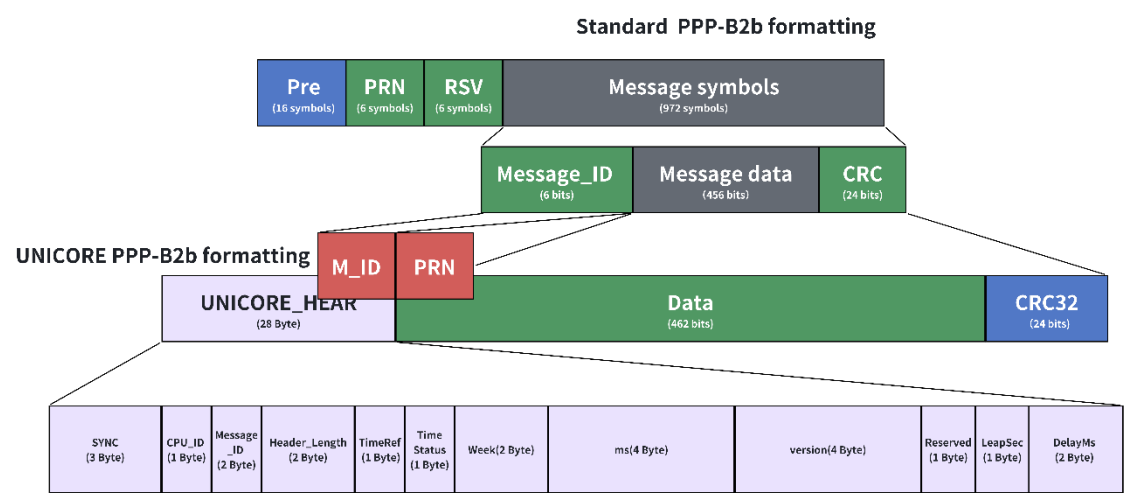


Fig Mapping between the standard PPP-B2b message framing and the Unicare receiver output format.

(2) PPP-B2b IOD consistency control with keep-last-valid update gating

The PPP-B2b ICD links contents across different information types using IOD (Issue of Data) indicators to ensure that orbit, clock, and bias corrections are used as a consistent set under the same data version. The relevant indicators include IOD_SSR (SSR data version), IODP (orbit-parameter version), IODN (navigation-message/ephemeris version), and IOD_Corr (version linkage among correction terms). RTKLIB-B2b follows this ICD concept and implements an IOD-consistency gating strategy: newly received correction components are maintained in raw->nav.B2bssr, while the solver state nav->B2bssr always keeps the last valid correction set that has passed completeness and consistency checks and is ready for PPP. The solver updates nav->B2bssr only when a new complete and IOD-consistent set becomes available; otherwise, it continues to use the last valid set, preventing partially updated corrections caused by asynchronous broadcasting or data gaps from contaminating the PPP solution.

At the information-type linkage level, each update cycle is anchored by the mask message

(Msg1). RTKLIB-B2b records the cycle IOD_SSR and IODP from Msg1 and derives the target satellite set from the mask. Orbit (Msg2), code-bias/offset (Msg3), and clock (Msg4) messages received in the same cycle are required to carry the same IOD_SSR as the current Msg1 context; Msg4 is additionally required to carry the same IODP as the current Msg1 context. Messages that do not satisfy these IOD requirements do not trigger an update of nav->B2bssr, and the solver continues to use the last valid correction set.

For per-satellite internal consistency, the decoded orbit and clock blocks carry their own IOD_Corr indicators (implemented as iodcorr[0] and iodcorr[1]). Only when these two values are identical are the orbit and clock terms considered to belong to the same version and allowed to enter the valid correction set stored in nav->B2bssr. When they differ, the new corrections for that satellite are treated as not yet complete, and the previously valid orbit and clock terms in nav->B2bssr are kept until a consistent pair is later formed.

For consistency with broadcast ephemeris, the orbit correction includes IODN, indicating the navigation-message/ephemeris version to which the correction applies. RTKLIB-B2b requires iodn to be consistent with the version identifier of an available broadcast ephemeris (in the implementation, eph.iode or an equivalent tag). If no consistent ephemeris is found, the update is treated as not corresponding to the current ephemeris and is not written into the valid correction set stored in nav->B2bssr, avoiding an ephemeris–correction version mismatch.

Finally, a solver-stage recheck is applied during PPP computation. The solver selects a broadcast ephemeris consistent with B2bssr->iodn. If no matching ephemeris is available, or if iodcorr[0] and iodcorr[1] are still found to be inconsistent at runtime, the PPP-B2b correction for that satellite is not applied in the current epoch. Processing then continues with the last valid correction set in nav->B2bssr, with an optional fallback to broadcast ephemeris only when necessary. This combination of decoder-side update gating and solver-side recheck improves robustness during data gaps and version transitions.

(3) Observed GEO PRN Anomaly (C61/C62) and Processing Strategy

During our experiment period (DOY 080–082, 2025), the PPP-B2b signals observed in the receiver outputs were associated with GEO PRNs C59, C60, and C62, while no PPP-B2b signals from C61 were observed. This discrepancy from the official PPP-B2b ICD (which lists

C59/C60/C61 as service satellites) is supported by multiple objective checks: (1) our monitoring logs showed no C61 PPP-B2b messages during the entire period, and C61 was also absent from the broadcast ephemeris products available to us at that time; (2) publicly available iGMAS constellation status information for the same period indicated C61 in a “no data” status and flagged C62 as “Unhealthy” (http://www.igmas.org/Gnss/Xzztpg/wxzt/cate_id/15.html), consistent with our observations of missing/unreliable signals; and (3) an independent technical note by Takahashi (Hiroshima City University) reported that among C59/C60/C61/C62, the latter two were in testing status (<https://s-taka.org/en/beidou-b2b-part2/>). Given this empirical behavior during the experiment period, RTKLIB-B2b includes a hard-coded decoder rule to skip PPP-B2b updates from specified GEO PRNs when necessary (e.g., PRN 62 in the present dataset) to avoid introducing potentially unreliable corrections into the PPP solution. Users who wish to change this behavior can modify the corresponding receiver-decoder source code. An example implementation in the Unicore decoding path is shown in Figure, where PPP-B2b updates from GEO PRN 62 are explicitly skipped. The ComNav decoder applies an analogous PRN-based exclusion rule in its corresponding module.

```

C unicore.c x
src > rcv > C unicore.c
299 {
301     double m_time_ep[8];
302     gtime_t m_time;
303     char satstr[10];
304     double c0;
305
306     PACKED_PPPB2BIN04 data4 = {0};
307     memcpy(&data4, raw->buff + UNICORE_LEN, sizeof(data4));
308
309     raw->geoprnr = data4.Prnr - 161 + 1;
310
311     if (raw->geoprnr == 62) {
312         raw->num_PPPB2BIN04--;
313         char time_str[128];
314         time2str(raw->time, time_str, 3);
315         trace(22, "Skipping PRN 62 at time %s: mes_type = 4 \n", time_str);
316         return 0;
317     }
318 }

```

Fig Example of hard-coded GEO PRN exclusion in the Unicore decoder (PRN 62)

(4) In-house Data Acquisition Platform

The datasets used in this project were collected using an in-house integrated data acquisition platform built by the authors. A Raspberry Pi 4B serves as the embedded computing core, on which RTKLIB-B2b is executed for real-time data reception, decoding, and PPP-B2b-aided PPP processing. The GNSS receiver OEM boards are connected to the Raspberry Pi through a custom-

designed interface/transfer board. This transfer board provides regulated power supply to the OEM board and bridges the receiver's serial (UART) output to the Raspberry Pi GPIO header for data acquisition. In our experiments, the Raspberry Pi reads the receiver output stream via the serial interface and forwards it to the RTKLIB-B2b processing pipeline (including optional logging). A representative view of the integrated platform and its enclosure is shown in Figure. Since the OEM boards are operated within this integrated platform (including power delivery and the receiver-interface/data-acquisition chain), we cannot fully rule out that platform-level factors—such as power-supply stability, grounding/noise coupling, or serial-interface handling—may affect the effective observation quality under certain configurations.

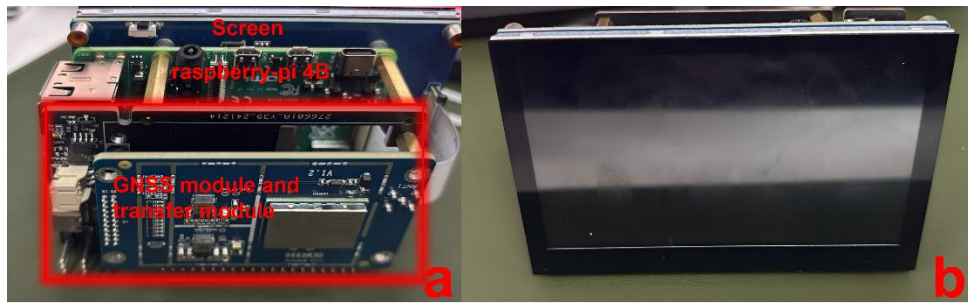


Fig. In-house integrated data acquisition platform used in this study

In addition, receiver-side OEM configuration options may affect the measurement characteristics reported at the output interface. For example, internal pseudorange smoothing (e.g., carrier-smoothed code) and related tracking or measurement-filter settings can change the noise and multipath-like signatures of the pseudorange observable, thereby influencing MP statistics computed from the logged data. In our experiments, both OEM receivers were operated with vendor-default internal settings, and we did not explicitly verify whether such smoothing or filtering options were enabled. Therefore, the observation-quality comparisons (especially MP-based results) should be interpreted as specific to the present platform, receiver configuration, and experimental setup, and would benefit from further controlled tests with explicitly documented receiver settings.

(5) Reference Commands for Receiver Observation, Ephemeris, and PPP-B2b Data Output

This section provides reference command sequences for configuring ComNav and Unicore receivers to output the necessary data streams for observation processing, ephemeris collection, and PPP-B2b corrections via specified communication ports

(COM1, COM2, COM3).

a. SINO

The following sequence configures a Sino receiver:

```
# Clear existing logs on COM ports
Unlogall COM1
Unlogall COM2
Unlogall COM3

# Log RTCM SSR and Observation messages on COM1
Log com1 rctm1074b ontime 5
Log com1 rctm1084b ontime 5
Log com1 rctm1094b ontime 5
Log com1 rctm1124b ontime 5
# Log RTCM Ephemeris messages on COM1 (adjust interval as needed)
Log com1 rctm1019b ontime 90
Log com1 rctm1020b ontime 90
Log com1 rctm1045b ontime 90
Log com1 rctm1042b ontime 90

# Log Proprietary Ephemeris messages on COM2 (alternative/redundant)
Log com2 bd3ephemb ontime 1
Log com2 gpsephemb ontime 1

# Log Raw PPP-B2b Subframe data on COM3
Log com3 B2BRAVNAVSUBFRAMEb ontime 1

# Save the configuration
Saveconfig
```

b. UNICORE

The following sequence configures a Unicore Communications receiver:

```
# Clear existing logs on COM ports
unlog com1
unlog com2
unlog com3

# Log RTCM SSR messages on COM1 (Interval 1 second)
rctm1074 com1 1
rctm1084 com1 1
rctm1094 com1 1
rctm1124 com1 1

# Enable B2b-PPP mode
CONFIG PPP ENABLE B2b-PPP
```

```
# Log Proprietary PPP-B2b messages on COM2 (Output on change)
```

```
PPPB2BINFO1B COM2 ONCHANGED
```

```
PPPB2BINFO2B COM2 ONCHANGED
```

```
PPPB2BINFO3B COM2 ONCHANGED
```

```
PPPB2BINFO4B COM2 ONCHANGED
```

```
# Log Proprietary Ephemeris messages on COM3 (Interval 5 seconds)
```

```
GPSEPHB COM3 5
```

```
BD3EPHB COM3 5
```

```
# Save the configuration
```

```
saveconfig
```